Q1

Assume an array of 10 integers (each integer is a word, i.e. 4 bytes). A compiler associates variables **x** and **y** with registers **r0** and **r1**, respectively. Assume that the base address for the array is located in register **r2**.

a) Translate this C statement into ARM assembly.

```
x = array[5] + y;
```

```
LDR, R1, [R2, 20]
ADD R1, R1, R0
```

b) Translate this following C code into ARM assembly.

```
int sum = 0;
for (int i = 0; i < 10; i++)
    sum += array[i];
```

```
        AREA findSum, CODE
        EXPORT      __main
        ALIGN
        ENTRY
__main
        LDR R0, =size
        LDR R3, [R0]

        LDR R2, =numbers
        MOV  R0, #0         ; SUM
        MOV  R1, #0         ; Loop Index
        B    TEST
LOOP    LDR  R1, [R2]
        ADD  R0, R0, R1
        ADD  R2, R2, #4
        ADD  R1, R1, #1
TEST    CMP  R1, R3
        BLT  LOOP

STOP    B    STOP

        AREA myData, DATA
        ALIGN
numbers DCD  -1, 5, 3, 8, 10, 23, 6, 5, 2, -10
size    DCD  10
        END
```

Q2

Suppose each register has only <u>five</u> bits. **R0 = 0b11101** and **R1 = 0b10110,** what is the N, Z, C, V flags of the following instructions.

Assuming initially N = 0, Z = 0, C = 0, V = 0, and these instructions are executed independently.

a) ADDS R3, R0, R1

- `11101 (-3) + 10110 (-10) = 10011 (-13)`
- `N = 1, Z = 0, C = 1, V = 0`

b) SUBS R3, R0, R1

- `11101 (-3) - 10110 (-10) = 00111 (7)`
- `N = 0, Z = 0, C = 1, V = 0`

c) EOR R3, R0, R0

- `NOT EORS, thus no change`

d) ANDS R3, R1, R1, LSL #3

- `10110 & 10000 = 10000`
- `N = 1, Z = 0, C = 1 , V = 0`

Q3

Translate the following code into C program and explain what does it do.

```
f1    MOV R2, #1
      MOV R1, #1
loop  CMP R1, R0
      BGT done
      MUL R2, R1, R2
      ADD R1, R1, #1
      B   loop

done  MOV R0, R2
      MOV pc, lr
```

```
int f1(int R0){
    R2 = 1;
    for(R1 = 1; R1 <= R0; R1++){
        R2 = R1*R2;
    }
    return R2;
}
```

The program finds R0!

Q4

The table below provides the memory addresses of some of the NVIC registers.

Address	Name	Description
0xE000E100-0xE000E11C	NVIC_ISER0-NVIC_ISER7	Interrupt Set-Enable Registers
0xE000E180-0xE000E19C	NVIC_ICER0-NVIC_ICER7	Interrupt Clear-Enable Registers
0xE000E400-0xE000E4EC	NVIC_IPR0-NVIC_IPR59	Interrupt Priority Registers

Write an assembly code that performs the following three tasks:

a) enable the interrupt number 45, and

```
Enable Interrupt:
  LDR r0, =NVIC_ISER1 ;  0xE000E104 is also OK!
  LDR r1, [r0]
  ORR r1, 1 << 13
  STR r1, [r0]
```

(Hints: Each interrupt has one enable bit, one disable bit, and one byte of priority.)

b) sets its priority bits to 2, and

```
Priority Setting
   LDR r0, =NVIC_IPR11  ;   0xE000E42C is also OK!
   LDR r1, [r0]
   ORR r1, 2<<12        ; 2<<8 is also OK!
   STR r1, [r0]
```

c) disable the interrupt 45

```
Disable Interrupt
   LDR r0, =NVIC_ICER1 ;   0xE000E184 is also OK!
   LDR r1, [r0]
   ORR r1, 1<<13
   STR r1, [r0]
```